

Classificação de textos humanos e gerados por Inteligência Artificial: uma abordagem híbrida com TF-IDF + LinearSVC e BERT

Integrantes:

Carlos Augusto Freire Maia de Oliveira - RA: 21.00781-0

Cesar Augusto Bresciani Junior - RA: 21.00478-0

Enzo Leonardo Sabatelli de Moura - RA: 21.01535-0

Resumo

Este estudo propõe e avalia dois classificadores binários capazes de identificar a autoria textual: um modelo linear tradicional baseado em TF-IDF e um modelo de linguagem profunda, o BERTimbau. São utilizados textos de três domínios distintos: artigos da Wikipédia, redações do ENEM e *reviews* de produtos, balanceados entre exemplos humanos e gerados por IA. O desempenho foi validado por meio de *cross-validation*, com os resultados apontando uma “vitória” do BERTimbau, que supera os métodos tradicionais em acurácia e generalização, embora com maior custo computacional.

1. INTRODUÇÃO

Com a crescente sofisticação de sistemas generativos de texto, como Gemini e ChatGPT, torna-se essencial desenvolver métodos capazes de detectar automaticamente a autoria textual. Essa necessidade é especialmente relevante em contextos educacionais, acadêmicos e jornalísticos, nos quais a autenticidade da produção textual é um valor central.

Este trabalho busca avaliar e comparar duas abordagens de classificação textual: uma delas baseada em representações vetoriais clássicas (TF-IDF + LinearSVC) e outra baseada em Deep Learning (BERTimbau), aplicadas à detecção de autoria humana ou artificial.

2. OBJETIVO

O principal objetivo deste projeto é desenvolver e avaliar um sistema automatizado capaz de identificar se um texto foi escrito por um humano ou por uma Inteligência Artificial a partir da análise de padrões linguísticos, estilísticos e semânticos.

Mais do que apenas distinguir as origens textuais, o estudo busca comparar o desempenho de diferentes abordagens de modelagem de linguagem, utilizando tanto métodos tradicionais de Machine Learning (baseados em vetorização TF-IDF e classificação com o LinearSVC) quanto técnicas de Deep Learning (baseadas nos embeddings contextuais do modelo BERTimbau).

3. MATERIAIS E MÉTODOS

Três conjuntos foram criados com amostras humanas e textos gerados por IA:

Fonte	Tipo de escrita	Características
Wikipédia	Informativa	Linguagem formal e objetiva
ENEM	Acadêmica	Estrutura argumentativa e coesão
Mercado Livre Cotidiana		Linguagem natural, emoção e opinião

Para cada texto humano, foi também gerado um texto por meio de Inteligência Artificial a partir de diferentes prompts.

4. PRÉ-PROCESSAMENTO E LIMPEZA DE DADOS

A fim de garantir que os modelos aprendessem padrões de estilo e discurso, realizamos uma limpeza cuidadosa nos dados. Removemos títulos automáticos, URLs, marcas de

formatação e expressões típicas de IA (como “Neste artigo veremos...”). O objetivo era eliminar pistas fracas que poderiam ser usadas pelos modelos como atalhos para fazer a distinção, forçando-os a aprender as características mais profundas dos textos.

```
import re
def limpar_texto(txt):
    txt = re.sub(r"^[a-zA-ZÀ-ÿ0-9 ]", " ", txt)
    txt = re.sub(r"\s+", " ", txt).strip().lower()
    return txt
```

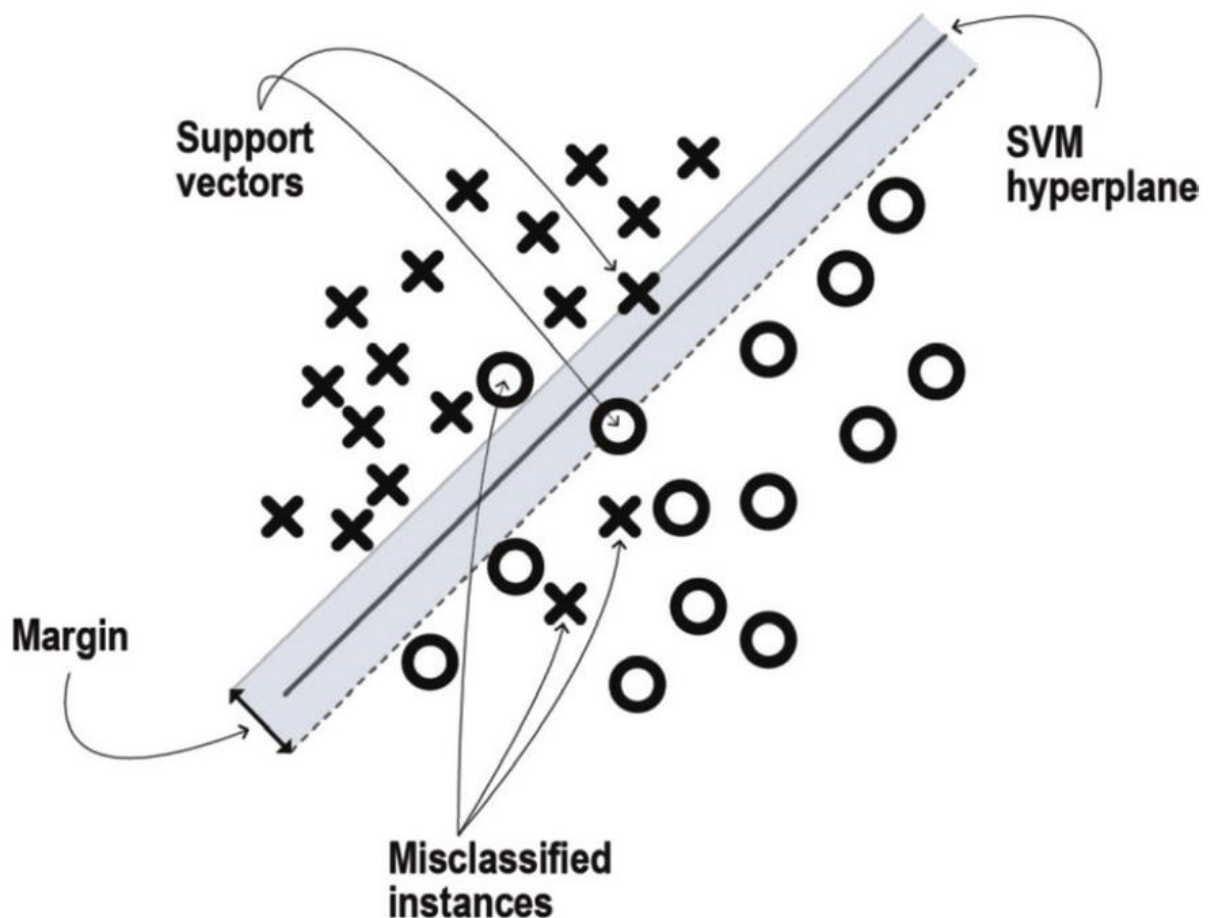
5. MODELOS E PARÂMETROS UTILIZADOS

5.1. VETORIZAÇÃO TF-IDF E MODELO LINEARSVC

Começamos com o modelo **TF-IDF** + **LinearSVC** como nossa linha de base. O TF-IDF é uma técnica de transformação de texto em números que possui duas partes:

- **TF (Term Frequency)**: mede a frequência com que uma palavra aparece em um texto.
- **IDF (Inverse Document Frequency)**: reduz o peso de palavras muito comuns ou muito raras e realça termos mais informativos.

Já o LinearSVC é um classificador baseado em máquinas de vetores de suporte (SVM) que tenta separar as classes (humano vs IA) com um hiperplano, maximizando a margem entre os pontos das diferentes classes. Essa abordagem funciona bem quando os vetores de TF-IDF já apresentam uma separação razoável entre as classes.



Utilizamos dos seguintes parâmetros na montagem do modelo:

```
vectorizer = TfidfVectorizer(  
    lowercase=True,  
    ngram_range=(1, 2), # unigrams + bigrams  
    max_df=0.9,  
    min_df=1,  
    max_features=20000,  
)
```

```
model_svc = LinearSVC(C=1.0)
```

- lowercase=True (default), para assegurar que palavras como "Texto" e "texto" sejam lidas igualmente
- ngram_range=(1,2), que faz o vetorizador trabalhar tanto com unigramas quanto bigramas
- max_df=0.9, que ignora palavras com frequência maior que 90% nos documentos, para evitar palavras com significado muito pequeno no modelo

- `min_df=1` (default), que faz com que todas as palavras que apareçam pelo menos uma vez nos documentos sejam consideradas na vetorização
- `max_features=20000`, que considera apenas as 20000 palavras mais frequentes nos documentos.

Além disso, para separação dos dados em teste e treino foram usados os seguintes parâmetros:

```
x_train, x_test, y_train, y_test = train_test_split(
    df["texto"], df["classe"], test_size=0.3, random_state=42, stratify=df["classe"]
)
```

`test_size=0.3`: 30% dos exemplos vão para o conjunto de teste, 70% para treino.

`random_state=42`: fixa a semente do gerador aleatório para que a divisão seja reprodutível.

`stratify=df["classe"]`: garante que a distribuição das classes no treino e no teste seja a mesma da distribuição original (importante se as classes são desbalanceadas).

Já na construção do próprio modelo `LinearSVC`, foi usado `C=1.0` (default). Essa constante é referente a "força de regularização" do modelo. Quanto maior o valor, mais ajustado aos dados de treino o modelo será (menos regularização). Quanto menor o valor, mais simples o modelo será (mais regularização). Essa constante foi mantida em 1.0 por ser um valor balanceado e padrão para todos os modelos.

```
model_svc = LinearSVC(C=1.0)
```

Como resultado, obtivemos as seguintes métricas:

```
Matriz de confusão [rows: humano, ia]:
[[81 20]
 [18 85]]

Classification report:
```

	precision	recall	f1-score	support
humano	0.8182	0.8020	0.8100	101
ia	0.8095	0.8252	0.8173	103
accuracy			0.8137	204
macro avg	0.8139	0.8136	0.8137	204
weighted avg	0.8138	0.8137	0.8137	204

O modelo LinearSVC obteve uma acurácia de 81.37% nos dados de teste. A matriz de confusão mostrou que o modelo acertou 81 exemplos humanos e 85 exemplos gerados por IA, com 20 falsas detecções de humanos e 18 falsas detecções de IA.

A classe "ia" teve recall ligeiramente superior (82.52%) em relação à classe "humano" (80.20%), e o modelo teve um bom equilíbrio no f1-score entre as classes, com 81.00% para "humano" e 81.73% para "ia".

Vale lembrar que a vetorização não é capaz de entender nuances e contextos de forma profunda. Por exemplo, a frase "O gato preto dorme sobre o tapete macio" seria igual à frase "Sobre o tapete macio dorme o gato preto" para o TF-IDF e consequentemente pelo LinearSVC, então nesta configuração não estão sendo analisados padrões de escrita.

Para mitigar isso, foram utilizados bigramas além dos unigramas para uma análise maior de contexto. Porém, mesmo assim, a análise de contexto não é profunda. Além disso, utilizar mais palavras na análise (como trigramas ou tetragramas) não alteram significativamente a acurácia do modelo e servem apenas para deixá-lo mais pesado e lento, como será visto na parte de **Cross-validation**.

5.2. MODELO BERT (BERTIMBAU + FINE-TUNING E DEEP-LEARNING)

O modelo BERT (Bidirectional Encoder Representations from Transformers) é um LLM pré-treinado que captura contextos mais profundos do texto, lendo sentenças como sequências e aprendendo representações contextuais das palavras. Isso permite que o modelo compreenda o significado de uma palavra dependendo do seu contexto, como no caso da palavra "banco", que pode ter significados diferentes em “banco do parque” e “banco financeiro”.

Ele possui capacidade para 110 milhões de parâmetros de entrada. Sendo assim, pode ser considerado um LLM “pequeno” ou “leve”, até mesmo em comparação com modelos como o Tiny LLaMa, que possui capacidade para 1.1 bilhões de parâmetros.

A fim de ampliar o vocabulário do nosso modelo, usamos um modelo pré-treinado com textos da língua portuguesa (BERTimbau) e aplicamos fine-tuning neste mesmo modelo.

A importação do modelo foi feita da seguinte forma:

```
tokenizer = BertTokenizer.from_pretrained("neuralmind/bert-base-portuguese-cased")
model_bert = BertForSequenceClassification.from_pretrained(
    "neuralmind/bert-base-portuguese-cased", num_labels=2
).to(device)
```

Os parâmetros utilizados no treino foram os seguintes:

```
MAX_LENGTH = 256
BATCH_SIZE = 4
EPOCHS = 5
LEARNING_RATE = 2e-5
```

`MAX_LENGTH = 256`, é o limite máximo de tokens processados por texto. Foi reduzido de 512 para reduzir o uso de memória durante o treino

`BATCH_SIZE = 4`, que limita o número de elementos processados a cada passo de treinamento. Foi reduzida de 8, também para poupar memória.

`EPOCHS = 5`, que é o número de passagens completas pelos dados de treino. 5 épocas foram suficientes para mitigar bastante a perda enquanto evita instabilidades e mantém o tempo de treino aceitável.

`LEARNING_RATE = 2e-5`, taxa de aprendizado para o otimizador durante o fine-tuning. $2e-5$ é considerado um ponto de partida equilibrado para o modelo BERT. Aumentar muito o Learning Rate pode causar instabilidades no treinamento, porém deixá-lo muito baixo faz com que o modelo não aprenda tão rápido.

Para separar os dados de teste e treino, foram usados os mesmos parâmetros do modelo linear.

Como resultado, obtivemos o seguinte:

```

Classification Report:
              precision    recall  f1-score   support

    humano      1.0000      0.9109      0.9534        101
         ia      0.9196      1.0000      0.9581        103

 accuracy              0.9559        204
 macro avg      0.9598      0.9554      0.9558        204
weighted avg      0.9594      0.9559      0.9558        204

Confusion Matrix [humano, ia]:
[[ 92   9]
 [  0 103]]

```

Assim, é evidente que o modelo BERT com fine-tuning, apesar de computacionalmente caro, foi capaz de atingir uma grande precisão nos dados de teste, com quase 96% de acurácia, e 100% de acertos em casos de humanos.

6. CROSS-VALIDATION

A fim de detectar se algum dos modelos (ou os dois) sofrem de over-fitting (um sobreajuste ou “viés” aos dados de teste), foi feito o cross-validation com os dois modelos.

O cross-validation divide o dataset em conjuntos aleatórios de dados, e executa o treinamento e teste em cada um deles. Ao final, a acurácia média dos testes pode ser medida para uma verificação mais precisa da eficácia de cada modelo para predições em novos conjuntos de dados.

Para o modelo linear, obtivemos os seguintes resultados no cross-validation:

```

Cross-validation results:
Mean accuracy: 0.7557 (+/- 0.1571)
Individual fold scores: [0.69117647 0.75735294 0.64705882 0.83088235 0.85185185]

```

Observe que a acurácia varia de 65% até 85%. Isso indica que o modelo é facilmente influenciado pelos dados de treinamento e teste, evidenciando que ele sofre de over-fitting.

Além disso, foram testados diferentes valores para os conjuntos de palavras, desde bigramas até pentagramas, onde i indica o número máximo dos conjuntos de palavras (por exemplo, $i = 2$ indica que o modelo está usando tanto unigramas quanto bigramas).


```
Cross-validation results (i = 2):  
Mean accuracy: 0.7557 (+/- 0.1571)  
Individual fold scores: [0.69117647 0.75735294 0.64705882 0.83088235 0.85185185]  
  
Cross-validation results (i = 3):  
Mean accuracy: 0.7571 (+/- 0.1635)  
Individual fold scores: [0.68382353 0.76470588 0.64705882 0.83088235 0.85925926]  
  
Cross-validation results (i = 4):  
Mean accuracy: 0.7601 (+/- 0.1676)  
Individual fold scores: [0.68382353 0.77205882 0.64705882 0.83088235 0.86666667]  
  
Cross-validation results (i = 5):  
Mean accuracy: 0.7616 (+/- 0.1764)  
Individual fold scores: [0.68382353 0.75735294 0.64705882 0.84558824 0.87407407]
```

Observa-se então que aumentar o número de palavras em cada conjunto não compensa na melhoria de acurácia do modelo. Além disso, aumentar esse número excessivamente faz com que o modelo se torne mais computacionalmente exigente.

Para o modelo BERT também foi implementado o cross-validation. Neste caso, o cross-validation é um algoritmo exaustivo, pois é composto de 5 iterações de treinamento em um modelo que já usa 5 épocas. Assim, são necessárias 25 iterações de treino totais, o que pode tomar um tempo considerável a depender da máquina utilizada.

Com o cross-validation do modelo BERT, obtiveram-se os seguintes resultados:

```
Cross-validation results:  
Mean accuracy: 0.9411 (+/- 0.0404)  
Individual fold scores: [0.9338235294117647, 0.9558823529411765, 0.9558823529411765, 0.9044117647058824, 0.9555555555555556]
```

O modelo sofreu variações mínimas na acurácia média, de 90% até 96%. Sendo assim, podemos dizer que o modelo não sofreu de over-fitting, atingindo resultados muito parecidos com diferentes conjuntos de treinamento/teste.

7. RESULTADOS E CONCLUSÕES

Pode-se concluir então que o modelo LinearSVC obteve um desempenho geral satisfatório, com 76% de acurácia média entre as 5 iterações do cross-validation. Ele funcionou especialmente bem com textos mais estruturados. Porém, apresentou dificuldade em reconhecer nuances de escrita humana mais naturais ou emocionais, especialmente nos reviews. Também foi observado que o modelo sofre de over-fitting, variando bastante em acurácia de acordo com os dados utilizados para treinamento/teste.

O modelo BERT superou por muito o modelo tradicional. Ele atingiu aproximadamente 94% de acurácia média. Além disso, foi extremamente preciso em detectar textos gerados por

IA. Isso acontece pois o BERT é capaz de capturar padrões discursivos e não apenas o vocabulário. Por fim, é um modelo que se mostra robusto ao não sofrer de over-fitting, mostrando que não depende dos dados utilizados em treinamento/teste para sua precisão.

8. APLICAÇÃO WEB INTERATIVA

Para tornar o projeto mais acessível e interativo, desenvolvemos uma aplicação web que permite aos usuários tanto inserir qualquer texto e obter a probabilidade de ser IA ou Humano quanto participar de um jogo de adivinhação, tentando identificar se o texto foi escrito por um humano ou uma IA.

Reconhecedor de Textos

TEXTOS

JOGO

Digite ou cole seu texto aqui

esse texto é gerado por um humano!

ANALISAR TEXTO

Resultados:

Modelo: **BERTimbau**

Predição: **HUMANO**

Confiança: 99.83%

Modelo: **TF-IDF + LinearSVC**

Predição: **IA**

Confiança: 72.61%

Reconhecedor de Textos

TEXTOS

JOGO

A gola é um pouco apertada quase não consigo passar a cabeça mas e ótimo o produto.

IA

HUMANO

Texto 1 de 25